

Препроцессоры HTML

Препроцессоры

Препроцессором называют инструмент преобразования одного синтаксиса в другой в рамках какого-либо языка программирования или разметки.

Их использование позволяет сократить время на написание кода, а сам код автоматически привести к определенному виду.

Также препроцессоры способны заменить или даже расширить функционал некоторых конструкций языка.

Препроцессоры HTML

Препроцессор **HTML** использует свой синтаксис разметки страницы. После сборки его код преобразуется в HTML разметку.

Примеры препроцессоров HTML:

- Haml – разработан Hampton Catlin на языке Ruby;
- Pug (переименован , раньше назвался JADE) – написан на языке JavaScript ;
- Slim.

Препроцессоры HTML

Стандартный рабочий процесс с использованием препроцессоров HTML подразумевает использование какой-либо инструмента сборки (например, Gulp, webpack, которые полностью автоматизирует компиляцию HTML из Pug).

Актуальность использования препроцессоров зависит от объёма и характера задач. На более-менее крупных проектах он позволяет добиться серьезного прироста скорости благодаря упрощённому синтаксису и дополнительным возможностям (миксины, шаблоны и пр.)

Преппроцессор Pug

Pug – это преппроцессор Html, написанный на языке JavaScript для Node.js. После интерпретации сервером синтаксис Pug превращается в Html код.

Пример. Выведем на страницу «Hello, World!».

Код Pug:

```
Html(lang=«ru»)  
  head  
    meta(charset=«utf-8»)  
    title= «Пример Pug»  
  body  
    h1 Hello, World!
```



Код HTML:

```
<html lang="ru">  
  <head>  
    <meta charset="utf-8"/>  
    <title>Пример Pug</title>  
  </head>  
  <body>  
    <h1>Hello, World!</h1>  
  </body>  
</html>
```

Комментарии

Однострочные комментарии начинаются с символов «//». Такие комментарии отображаются и в коде Pug, и в HTML-коде.

Код Pug:

```
// Комментарий
```

Код HTML:

```
<!-- Комментарий -->
```

Однострочные комментарии, которые не видны в html коде начинаются с символов «//-»

Код Pug:

```
// - Комментарий, в HTML не выводится
```


Комментарии

Многострочные комментарии оформляются так:

- сначала указывается «//» или «//-»;
- затем с новой строки с отступом текст многострочного комментария.

Код Pug:

```
// - Комментарий  
    включает несколько строк  
    в html-код не выводится  
//  Комментарий  
    включает несколько строк  
    в html-код выводится
```

Код HTML:

```
<!--  Комментарий  
включает несколько строк  
в html-код выводится -->
```

Теги

Синтаксис описания тегов в Pug:

- записываются без угловых скобок и закрывающих тегов;
- содержимое тега записывается после тега через пробел;
- если текст слишком длинный, его можно поместить на отдельных строках, для этого после тега нужно поставить точку (больше никаких символов быть не должно), а затем начать текст с новой строки с отступом;
- внутри теста можно использовать обычные теги;
- если тег включает вложенные теги – они записываются с отступом (Таб или пробел).

Теги

Пример. Создадим абзац текста.

Код Pug:

`p <b>Pug</b> – это шаблонизатор Html.`

или

`p.`

`<b>Pug</b> – это шаблонизатор Html.`

Код HTML:

`<p><b>Pug</b> – это шаблонизатор Html.</p>`

Теги

Пример. Создадим нумерованный список

Код Pug:

`p Языки программирования:`

`ol`

`li Python`

`li JavaScript`

`li Java`

`li C#`

Код HTML:

`<p>Языки программирования:</p>`

`<ol>`

`<li>Python</li>`

`<li>JavaScript</li>`

`<li>Java</li>`

`<li>C#</li>`

`</ol>`

Атрибуты тегов

Синтаксис описания атрибутов тегов:

- задаются в круглых скобках после названия тегов;
- если атрибутов несколько, они задаются либо через запятую, либо через пробел;
- допускается запись атрибутов в несколько строк.

Атрибуты тегов

Код Pug:

```
input(type="text" name="username" placeholder="Фамилия И.О.")
```

или

```
input(  
  type='text'  
  name='username'  
  placeholder="Фамилия И.О."  
)
```

Код HTML:

```
<input type="text" name="username" placeholder="Фамилия И.О.">
```

Теги и атрибуты

Задание 1. Написать код Pug, чтобы после его трансляции получился следующий HTML-код

```
<ol type="I">
  <li> Препроцессоры HTML
    <ul type="disc">
      <li><a href="#" >Haml</a></li>
      <li>Pug</li>
    </ul>
  </li>
  <li> Препроцессоры CSS
    <ul type="circle">
      <li>SAAS</li>
      <li>Stylus</li>
    </ul>
  </li>
</ol>
```

Описание классов

Синтаксис описания классов:

- классы записываются после тега через точку;
- допускается перечисление нескольких классов, перед каждым классом ставится точка;
- классы можно описать как обычный атрибут тега;
- атрибуты тега указываются ПОСЛЕ описания классов.

Описание классов

Пример. Отнесем тег `<p>` к классу **text**, а тег `<ul>` к классу **list**.

Код Pug:

```
p.text Языки программирования:
ol.list(start="2")
  li Python
  li JavaScript
  li Java
  li C#
или
p(class="text") Языки программирования:
ol(class="list" start="2")
  li Python
  li JavaScript
  li Java
  li C#
```

Код HTML:

```
<p class="text">Языки
  программирования:
</p>
<ol class="list" start="2">
  <li>Python</li>
  <li>JavaScript</li>
  <li>Java</li>
  <li>C#</li>
</ol>
```

Идентификаторы

Синтаксис описания идентификаторов:

- идентификаторы записываются после тега через символ «#»;
- идентификаторы можно описать как обычный атрибут тега;
- одному тегу можно и классы, и идентификаторы (рекомендуется идентификаторы записывать после классов);
- атрибуты тега указываются ПОСЛЕ описания классов и идентификаторов.

Идентификаторы

Пример. Присвоим тегу `<p>` идентификатор **text**, а тег `<ul>` отнесем к классу **list** и еще дадим имя **lang**.

Код Pug:

```
p#text Языки программирования:  
ol.list#lang(start="2")
```

```
  li Python  
  li JavaScript  
  li Java  
  li C#
```

или

```
p(id="text") Языки программирования:  
ol(class="list" id="lang" start="2")
```

```
  li Python  
  li JavaScript  
  li Java  
  li C#
```

Код HTML:

```
<p id="text">Языки  
  программирования:</p>  
<ol class="list" id="lang"  
  start="2">  
  <li>Python</li>  
  <li>JavaScript</li>  
  <li>Java</li>  
  <li>C#</li>  
</ol>
```

Классы и идентификаторы

Задание 2. Написать код Pug, чтобы после его трансляции получился следующий HTML-код

```
<ol id="list" type="I">  
  <li> Препроцессоры HTML  
    <ul type="disc">  
      <li class="first"><a href="#"> Haml</a></li>  
      <li>Pug</li>  
    </ul>  
  </li>  
  <li> Препроцессоры CSS  
    <ul type="circle">  
      <li class="first">SAAS</li>  
      <li>Stylus</li>  
    </ul>  
  </li>  
</ol>
```

Переменные

В коде Pug можно использовать переменные и задавать им значения с помощью операторов присваивания:

- `var` переменная = значение ;

Переменные

Особенности использования переменной:

- Для того, чтобы эта строка не отображалась в HTML-коде, перед ней вставляется знак «-».
- Далее эту переменную можно использовать в коде, указав ее название.
- Тип данных переменной определяется по присвоенному ей значению.
- С переменной допустимы операции в соответствии с ее типом.
- Чтобы вставить переменную в текст тега, ее заключают в `#{ }`.
- Чтобы использовать переменную для обозначения тега внутри другого тега, ее заключают `#[ #{ } ]`

Переменные

Пример. Создадим тег с приветствием, в котором будет меняться имя и тег, которым это имя выделяется.

Код Pug

```
- var name = 'Алина'  
h4 Привет, меня зовут #{name}!
```

Код HTML

```
<h4>Привет, меня зовут Алина!</h4>
```

Переменные

Пример. Создадим тег с приветствием, в котором будет меняться имя и тег, которым это имя выделяется.

Код Pug

```
- var name = 'Алина'  
- var tag = 'em'
```

```
#{tag} #{name}!
```

Код HTML

```
<em>Алина!</em>
```

Переменные

Пример. Создадим тег с приветствием, в котором будет меняться имя и тег, которым это имя выделяется.

Код Pug

```
- var tag = 'em'  
- var name = 'Алина'  
h4 Привет, меня зовут #{tag} #{name}!
```

Код HTML

```
<h4>Привет, меня зовут  
  <em>Алина</em>!  
</h4>
```

Переменные

Пример. Создадим тег с приветствием, в котором будет меняться имя и тег, которым это имя выделяется.

Код Pug

```
- var tag = 'em'  
- var name = 'Алина'  
h4 Привет, меня зовут #{tag} #{name}!
```

Код HTML

```
<h4>Привет, меня зовут  
  <em>Алина</em>!  
</h4>
```


Переменные

Пример. Создадим тег с приветствием, в котором будет меняться имя и тег, которым это имя выделяется.

Код Pug

```
- var tag = 'em'  
- var name = 'Алина'  
h4 Привет, меня зовут #{tag} #{name}!  
- name += name  
h4 Привет, меня зовут #{tag} #{name}!
```

Код HTML

```
<h4>Привет, меня зовут  
  <em>Алина</em>!  
</h4>  
<h4>Привет, меня зовут  
  <em>АлинаАлина</em>!  
</h4>
```

Переменные

Пример. Создадим тег с приветствием, в котором будет меняться имя и тег, которым это имя выделяется.

Код Pug

```
- var tag = 'em'  
- var name = 'Алина'  
h4.  
  Привет, меня зовут  
  #{tag} #{name}!
```

Код HTML

```
<h4>Привет, меня зовут  
  <em>Алина</em>!  
</h4>
```

Переменные

Задание 3. Даны переменные

- `var tag = 'h';`
- `var i = 1;`

создать с их помощью следующий фрагмент HTML-кода:

```
<h1>Уровень 1</h1>
<h2>Уровень 2</h2>
<h3>Уровень 3</h3>
<h4>Уровень 4</h4>
```

Цикл each

Цикл **each** в Pug позволяют формировать теги на основе значений из списка.

Синтаксис цикла:

```
each переменная in [список]
... теги
    тег= переменная
...
```

Текстовое содержимое тега располагается через пробел, чтобы вместо текста в качестве содержимого тега задать значение переменной – **вместо пробела ставится знак «=»**.

Цикл each

Цикл **each** в Pug позволяют формировать теги на основе значений из списка.

```
each переменная in [список]
... теги
    тег= переменная
    ...
```

Выполняется этот оператор так:

1. переменной присваивается первое значение из списка;
2. значение переменной подставляется в какой-то тег внутри цикла
3. переменной присваивается следующее значение, повторяется пункт 2 до тех пор, пока все значения из списка не будут исчерпаны.

Цикл each

Пример. Создадим меню в блоке **nav**.

Код Pug:

```
nav.menu
  ul.list
    each name in ['Python', 'JavaScript', 'Java', 'C#']
      li
        a(href="#")= name
```

Код HTML:

```
<nav class="menu">
  <ul class="list">
    <li><a href="#">Python</a></li>
    <li><a href="#">JavaScript</a></li>
    <li><a href="#">Java</a></li>
    <li><a href="#">C#</a></li>
  </ul>
</nav>
```

Цикл each

Список в цикле **each** можно заменить на словарь с ключами, тогда в теле цикла будут доступны и ключи, и их значения:

```
each ключ, значение in {'значение' : 'ключ', ...}  
... теги  
    тег= ключ или значение  
...
```

Цикл each

Ключ или значение можно использовать как часть значения какого-то атрибута (это строка текста), тогда ключ или значение соединяются с другими строками через знак «+».

```
тег(атрибут="часть значения" + ключ или значение +  
"часть значения")  
...
```

Также включить значение переменной в текстовую строку можно, заключив переменную в #{ }:

```
тег(атрибут="часть значения #{ ключ или значение }  
часть значения")
```

Цикл each

Пример

Код Pug:

```
nav.menu
  ul.list
    each key, value in { 'Python' : 'lang1' ,
'JavaScript': 'lang2', 'Java' : 'lang3', 'C#': 'lang4' }
      li
        a(href= key + ".html")= value
```

Код HTML:

```
<nav class="menu">
  <ul class=" list">
    <li><a href="lang1.html">Python</a></li>
    <li><a href="lang2.html">JavaScript</a></li>
    <li><a href="lang3.html">Java</a></li>
    <li><a href="lang4.html">C#</a></li>
  </ul>
</nav>
```

Цикл each

Задание 4. Написать код Pug, чтобы после его трансляции получилась следующая таблица (предварительно создать необходимый список).

Товар	Цена
ручка	30.50
карандаш	50.00
альбом	156.00
тетрадь	21.50
ластик	10.20

Цикл for

Цикл **for** в Pug позволяет формировать некоторое количество тегов в соответствии с ограничениями.

Синтаксис цикла:

- for (инициализация управляющей переменной;
условие завершения;
изменение управляющей переменной)
...

Цикл while

Пример. Создадим таблицу квадратов первых 10 натуральных чисел.

Код Pug:

```
table(border="1")
  tr
    th Число
    th Квадрат
  - for (var i = 1; i <= 10; i++)
    tr
      td= i
      td= i * i
```

Код HTML:

```
<table border="1">
  <tr>
    <th>Число</th>
    <th>Квадрат</th>
  </tr>
  <tr>
    <td>1</td>
    <td>1</td>
  </tr>
  ...
  <tr>
    <td>10</td>
    <td>100</td>
  </tr>
</table>
```

Цикл while

Цикл **while** в Pug позволяет формировать последовательность тегов в зависимости от условия.

Синтаксис цикла:

```
while условие
  ...
```

Теги формируются до тех пор, пока условие – истина.

Цикл while

Пример. Создадим таблицу квадратов первых 10 натуральных чисел.

Код Pug:

```
- var i = 1;
table(border="1")
  tr
    th Число
    th Квадрат
  while i <= 10
    tr
      td= i
      td= i * i
    - i++
```

Код HTML:

```
<table border="1">
  <tr>
    <th>Число</th>
    <th>Квадрат</th>
  </tr>
  <tr>
    <td>1</td>
    <td>1</td>
  </tr>
  ...
  <tr>
    <td>10</td>
    <td>100</td>
  </tr>
</table>
```

Условный оператор

В коде Pug можно использовать условный оператор. Его синтаксис:

```
if условие
  ...
else
  ...
```

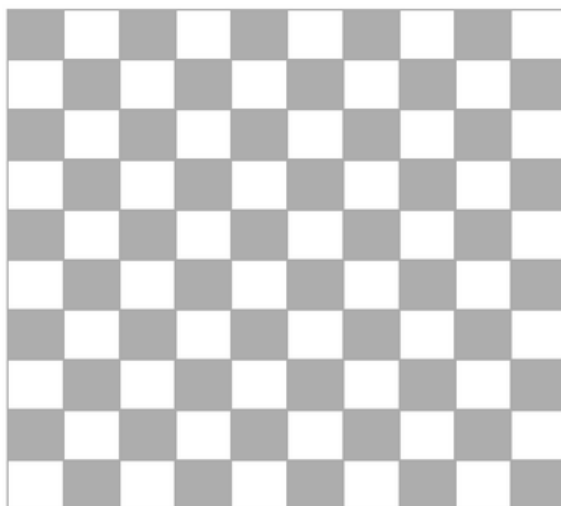
Пример:

```
- var gender= "м"
- var name= "Иван"
```

```
if gender == "м"
  p Уважаемый, #{name}!
else
  p Уважаемая, #{name}!
```

Циклы, условия

Задание 5. Сформируйте таблицу, размером **n** на **n**, содержащую ячейки двух классов (**black** и **white**). Ячейки отнести к классам, чтобы страница выглядела следующим образом (**n** задать как переменную):



Миксины

Миксины позволяют создавать многократно повторяемые блоки кода. Фактически - это функция в языке программирования.

Синтаксис объявления миксина:

```
mixin ИМЯ
  тело миксина
```

Синтаксис вызова миксина:

```
+ИМЯ
```

Код миксина можно вынести в отдельный файл, тогда в том документе, где он используется, этот файл необходимо подключить через **include** в верхней части документа.

Миксины

Пример. Создадим с помощью миксина список.

Код Pug:

```
mixin createList
  ol.list
    li Python
    li JavaScript
    li Java
    li C#
```

Код HTML:

```
<ol class="list">
  <li>Python</li>
  <li>JavaScript</li>
  <li>Java</li>
  <li>C#</li>
</ol>
```

Вызов миксина:

```
+createList
```

Миксины с параметрами

Синтаксис объявления миксина с параметрами:

```
mixin ИМЯ(параметр1, параметр2, ...)  
    тело миксина
```

Синтаксис вызова миксина:

```
+ИМЯ(фактический параметр1, фактический параметр2,  
...)
```

Миксины

Пример. Создадим с помощью миксина список, элементы списка передадим в качестве параметра.

Код Pug:

```
mixin createList(listLi)  
    ol.list  
        each name in listLi  
            li= name
```

Код HTML:

```
<ol class="list">  
    <li>Python</li>  
    <li>JavaScript</li>  
    <li>Java</li>  
    <li>C#</li>  
</ol>
```

Вызов миксина:

```
+createList(['Python', 'JavaScript', 'Java', 'C#'])
```

Миксины с параметрами

Для каждого параметра миксина можно установить значение по умолчанию, то есть значение, которое будет присвоено параметру, если при вызове миксина он пропущен:

```
mixin имя(параметр1=значение1,  
параметр2=значение2, ...)  
    тело миксина
```

Миксины с параметрами

Пример. Создадим с помощью миксина список, элементы списка передадим в качестве параметра, указав значение по умолчанию.

Код Pug:

```
mixin createList(listLi=['нет списка'])  
    ol.list  
        each name in listLi  
            li= name
```

Вызов миксина:

```
+createList
```

Код HTML:

```
<ol class="list">  
    <li>нет списка</li>  
</ol>
```


Миксины с условием

Пример. Создадим с помощью миксина список, элементы списка передадим в качестве параметра, укажем значение по умолчанию и вызовем миксин, пропустив параметр. Если параметр не передан – выведем не список, а абзац с текстом «нет списка».

Код Pug:

```
mixin createList(listLi='нет списка')
  if listLi == 'нет списка'
    p= listLi
  else
    ol.list
      each name in listLi
        li= name
```

Код HTML:

```
<p>нет списка</p>
```

Вызов миксина:

```
+createList
```

Миксины с параметрами

В качестве параметра миксина можно передать **атрибуты и их значения**. Атрибуты присоединяются к нужному элементу:

```
mixin имя(параметр1,...)
  ...
  элемент&attributes(attributes)
  ...
```

А при вызове список атрибутов указывается в скобках:

```
+имя(значение1, ..)(атрибут1="значение"
атрибут2="значение" ...)
```

Миксины с параметрами

Пример. Создадим с помощью миксина список, элементы списка передадим в качестве параметра. Передадим список атрибутов списка.

Код Pug:

```
mixin createList(listLi)
  ol.list&attributes(attributes)
    each name in listLi
      li= name
+createList(['Python', 'JavaScript', 'Java'])(start="2"
type="I")
```

Код HTML:

```
<ol class="list" start="2" type="I">
  <li>Python</li>
  <li>JavaScript</li>
  <li>Java</li>
```

Миксины

Задание 6. Создать с помощью миксина список. В качестве параметра передается тип списка, элементы списка и тег для выделения каждого элемента списка.

Вызов:

```
+createList('ul', ['Python', 'JavaScript', 'Java'],
'strong')
```

HTML код:

```
<ul>
  <li><strong>Python</strong></li>
  <li><strong>JavaScript</strong></li>
  <li><strong>Java</strong></li>
</ul>
```

Подключения

Pug позволяет подключать фрагменты кода, сохраненные в других файлах.

Например, можно отдельно создать файлы с кодом, описывающим **header**, **sidebar**, **content**, **footer**, а затем использовать их в различных файлах. Такой подход сделает код удобочитаемым и минимизирует исправления.

Синтаксис подключений:

`include имя_файла.pug`

В подключаемых файлах первый оператор начинается БЕЗ ОТСТУПА, все отступы выполняются относительно этого оператора. Отступ оператора `include` соответствует отступу вставляемого фрагмента.

Подключения

Пример. Создадим две страницы **header.pug** и **content.pug**, разместим их в папке **components**. Затем включим эти страницы в **index.pug**.

Код Pug:

Страница **header.pug**:

```
h1.title Языки программирования
```

Страница **content.pug**:

```
ol.list
  li Python
  li JavaScript
  li Java
  li C#
```

Страница **index.pug**:

```
doctype html
html(lang="ru")
  body
    //- Include header
    include components/header
    h2 Рейтинг
    //- Include content
    include components/content
```

Подключения

Код HTML:

```
<!DOCTYPE html>
<html lang="ru">
<head>
<body>
    <h1 class="title"> Языки программирования</h1>
    <h2> Рейтинг </h2>
    <ol class="list">
        <li>Python</li>
        <li>JavaScript</li>
        <li>Java</li>
        <li>C#</li>
    </ol>
</body>
</html>
```

Подключения

Также есть возможность подключать отдельные файлы стилей и скриптов:

```
// - создана папка css
style
    include css/style.css

// - создана папка JS
script
    include js/custom-script.js
```

Наследование шаблонов

Pug позволяет создавать шаблоны страниц, это позволяет любую страницу сайта создавать по заранее созданному

Наследование шаблона происходит через ключевое слово **extends**, далее через пробел указывается относительный путь до файла шаблона. Расширение файла *.pug указывать не обязательно.

Области шаблона, в которых необходимо вывести содержимое обозначаются словом **block** с указанием имени блока через пробел, например, **block header**.

Содержимое блока на страницах заменяется дочерним шаблоном, если там встречается этот блок.

Если блок на странице не встречается, этот блок вставляется в страницу из шаблона.

Наследование шаблонов

Пример. Создадим шаблон **template**, а затем его используем для создания двух страниц: **index** и **about**.

Шаблон Pug

```
// - template.pug
block variables
doctype html
html(lang="ru")
  head
    title #{title}
  body
    block header
      h1= title
    block content
    block footer
    p Copyright
```

Страницы Pug

```
// - index.pug
extends template
block variables
  - var title = 'Главная страница'
block content
  p Языки программирования:
  ul.list
    each name in ['Python',
                  'JavaScript', 'Java', 'C#']
    li= name
```


Наследование шаблонов

index.html

```
<html lang="ru">
  <head>
    <title>Главная страница</title>
  </head>
  <body>
    <h1>Главная страница</h1>
    <p>Языки программирования:</p>
    <ol>
      <li>Python</li>
      <li>JavaScript</li>
      <li>Java</li>
      <li>C#</li>
    </ol>
    <p> Copyright</p>
  </body>
</html>
```

блок **header** вставляется на страницу из шаблона, выводит на страницу заголовок, для которого использует значение переменной, определенных в блоке **variables**.

блок **content** заменяет пустой блок content шаблона

блок **footer** не встречается файле, поэтому он вставляется на страницу из шаблона

Наследование шаблонов

Пример. Создадим шаблон **template**, а затем его используем для создания двух страниц: **index** и **about**.

Шаблон Pug

```
// - template.pug
block variables
doctype html
html(lang="ru")
  head
    title #{title}
  body
    block header
      h1= title
    block content
    block footer
      p Copyright
```

Страницы Pug

```
// - about.pug
extends template
block variables
  - var title = 'О нас'
block content
  p Мы изучаем программирование.
```


Наследование шаблонов

about.html

```
<html lang="ru">
  <head>
    <title>0 нас</title>
  </head>
  <body>
    <h1>0 нас</h1>
    <p>Мы изучаем программирование.</p>
    <p> Copyright</p>
  </body>
</html>
```

блок **header** вставляется на страницу из шаблона, выводит на страницу заголовок, для которого использует значение переменной, определенных в блоке **variables**.

блок **content** заменяет пустой блок content шаблона

блок **footer** не встречается файле, поэтому он вставляется на страницу из шаблона

Наследование шаблонов

Содержимое блока шаблона полностью заменяется дочерним блоком, но можно добавить новое содержимое к уже имеющемуся, для этого используются операторы:

block append / prepend

block prepend - добавляет содержимое перед первым элементом блока.

block append - добавляет содержимое после последнего элемента блока.

Слово **block** можно опустить.

Наследование шаблонов

В нашем **примере** с помощью блока **footer** на страницу выводится абзац «Copyright». На главной странице после него выведем абзац «Любое копирование материалов сайта».

Страницы Pug

```
// - index.pug
extends template
block variables
  - var title = 'Главная страница'
block content
  p Языки программирования:
  ul.list
    each name in ['Python', 'JavaScript', 'Java', 'C#']
      li= name
block append footer
  p Любое копирование материалов сайта без разрешения
  владельца запрещено
```

Наследование шаблонов

index.html

```
<html lang="ru">
  <head>
    <title>Главная страница</title>
  </head>
  <body>
    <h1>Главная страница</h1>
    <p>Языки программирования:</p>
    <ol>
      <li>Python</li>
      <li>JavaScript</li>
      <li>Java</li>
      <li>C#</li>
    </ol>
    <p> Copyright</p>
    <p> Любое копирование материалов сайта без разрешения ... </p>
  </body>
</html>
```

Спасибо за внимание!